

ListView & RecyclerView

Building dynamic lists

by Ekramul Haque Amin

Lecturer, DIU

1

Lists

2

RecyclerView

3

ViewHolder

4

Canvas

5

2 Builds

Why Do We Need Lists?

Showing many similar items without hardcoding every row

A contact book, a chat, a news feed, a notes app — they all show **many items that share the same layout**.

Hardcoding a TextView for every contact doesn't scale — the list grows or shrinks at runtime, so the UI must be generated dynamically from data.



Think of it like index cards

Each row is a small card with the same template — a photo, a title, a subtitle.

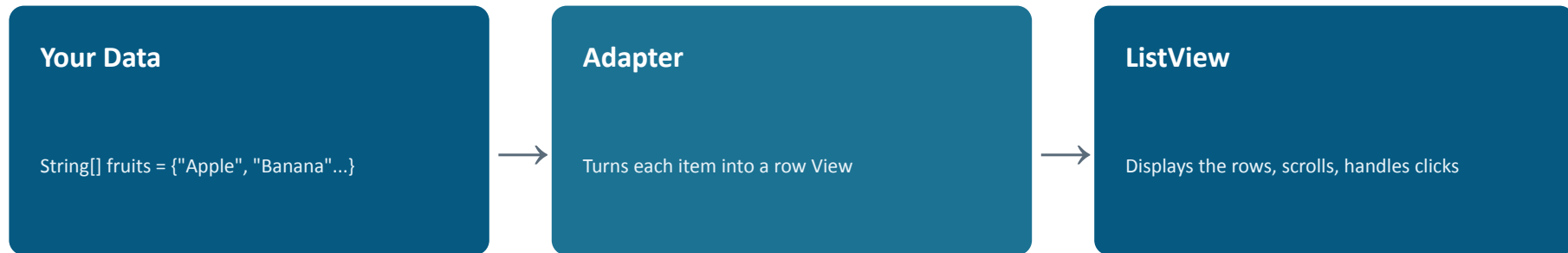
A list widget just repeats that template once per item in your data.

Today: two ways Android builds these repeating rows — ListView (simple) and RecyclerView (modern, flexible).

Introducing ListView

A scrollable list widget — connected to your data by an Adapter

ListView is a View that displays a vertical, scrollable list of rows. It doesn't know your data directly — it asks an **Adapter** to supply each row.



Two ways to connect data: `android:entries` in XML for a quick static list, or an `ArrayAdapter`/custom Adapter in Java for full control.

ListView with ArrayAdapter

The quickest way to show a list of simple text items

1 strings.xml — the data

```
<string-array name="listItems">

    <item>Orange</item>

    <item>Apple</item>

    <item>Banana</item>

</string-array>
```

2 MainActivity.java

```
String[] fruits = getResources()
    .getStringArray(R.array.listItems);
ArrayAdapter<String> adapter =
    new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_1,
        fruits);
ListView listView =
    findViewById(R.id.listView);
listView.setAdapter(adapter);
```

A Custom Row Layout & Adapter

For full control: an icon and two lines of text per row

list_item.xml

```
<ImageView
    android:id="@+id/icon" />
<TextView
    android:id="@+id/firstLine" />
<TextView
    android:id="@+id/secondLine" />
```

MyAdapter.java (extends BaseAdapter)

```
public View getView(int position,
    View convertView, ViewGroup parent) {
    if (convertView == null) {
        convertView = inflater.inflate(
            R.layout.list_item, null);
    }
    ((TextView) convertView
        .findViewById(R.id.firstLine))
        .setText(fruits[position]);
```

The Adapter Pattern, Explained

Every adapter must answer four questions

Method	Question It Answers
<code>getCount()</code>	How many items are in the list?
<code>getItem(position)</code>	What data object is at this position?
<code>getItemId(position)</code>	What unique ID identifies this position?
<code>getView(position, convertView, parent)</code>	What View should be drawn for this row?

Key idea: the list widget never touches your data directly. It always goes through these four questions — which is exactly why swapping `ListView` for `RecyclerView` later feels familiar.

Handling Clicks on List Items

Two listeners: a normal tap, and a long press



Tap — `setOnItemClickListener`

```
listView.setOnItemClickListener(  
    new AdapterView.OnItemClickListener() {  
        public void onItemClick(  
            AdapterView<?> parent, View v,  
            int position, long id) {  
            Toast.makeText(parent.getContext(),  
                "Selected " + fruits[position],  
                Toast.LENGTH_SHORT).show();  
        }  
    });
```



Long Press — `OnItemLongClickListener`

```
listView.setOnItemLongClickListener(  
    new AdapterView  
        .OnItemLongClickListener() {  
        public boolean onItemLongClick(  
            AdapterView<?> parent, View v,  
            int position, long id) {  
            // e.g. show a delete menu  
            return true;  
        }  
    });
```

ListView Extras: Fast Scroll & Section Indexer

Handy for long, alphabetically sorted lists



Fast Scroll

`android:fastScrollEnabled="true"` adds a thumb the user can drag to jump through a long list quickly.



Section Indexer

Implementing `SectionIndexer` shows an alphabet strip next to the scrollbar, jumping straight to a letter.

Both are entirely optional polish — useful for contact lists or dictionaries, not something every list needs.

From ListView to RecyclerView: Why Upgrade?

The modern, more flexible way to show lists



Better Performance

Reuses (recycles) row views as you scroll, and forces the efficient ViewHolder pattern — no accidental slow lookups.



Flexible Layouts

Swap LinearLayoutManager for GridLayoutManager or StaggeredGridLayoutManager without changing your adapter.



Built-in Animations

Item insert/remove/move animations come for free with notifyItem... calls.

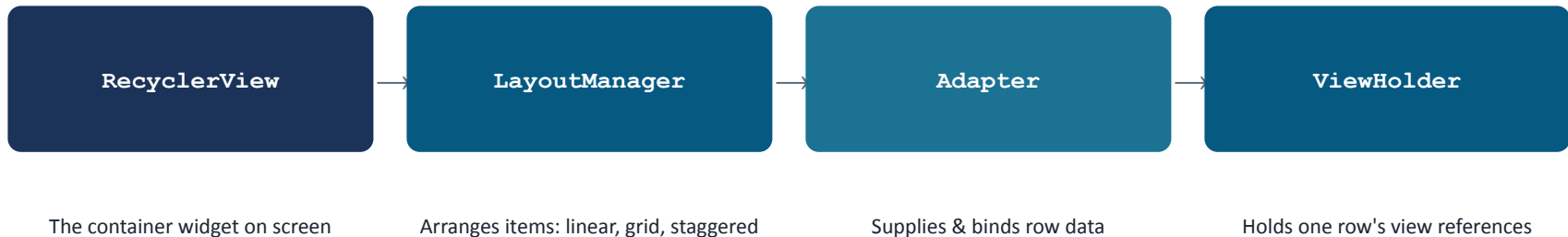


Decorations & Touch Helpers

ItemDecoration for dividers, ItemTouchHelper for swipe-to-delete and drag-to-reorder.

RecyclerView: The Core Pieces

Four parts that always work together



The flow: RecyclerView asks its LayoutManager where the next row goes, asks the Adapter to create or bind a ViewHolder for that position, and the ViewHolder's cached views get updated with data — all without a fresh `findViewById()` call every time you scroll.

Basic Setup: XML + Activity Code

Add the widget, then attach a LayoutManager and Adapter

activity_main.xml

```
<androidx.recyclerview.widget
    .RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

MainActivity.java

```
RecyclerView recyclerView =
    findViewById(R.id.recyclerView);
recyclerView.setLayoutManager(
    new LinearLayoutManager(this));
MyAdapter adapter = new MyAdapter(this);
recyclerView.setAdapter(adapter);
```

Building the Adapter Skeleton

Every RecyclerView.Adapter overrides these three methods

```
public class MyAdapter extends RecyclerView.Adapter<MyAdapter.MyViewHolder> {  
  
    @Override  
    public MyViewHolder onCreateViewHolder(ViewGroup parent, int viewType) { ... }  
  
    @Override  
    public void onBindViewHolder(MyViewHolder holder, int position) { ... }  
  
    @Override  
    public int getItemCount() { ... }  
}
```

The ViewHolder Pattern, Explained

Look up each row's views once — not every single scroll frame



Without a ViewHolder

Every scroll frame calls `findViewById()` again for the same row — wasted, repeated work that can make scrolling stutter.



With a ViewHolder

`findViewById()` runs once when the row is created; the ViewHolder object caches those references for reuse on every rebind.

```
public class MyViewHolder extends RecyclerView.ViewHolder {  
    TextView title;  
    public MyViewHolder(View itemView) {  
        super(itemView);  
        title = itemView.findViewById(R.id.text1); // looked up ONCE  
    }  
}
```

onCreateViewHolder() vs onBindViewHolder()

Two very different jobs, easy to mix up

Method	When It Runs	What It Does
<code>onCreateViewHolder()</code>	Only when a new row view is needed	Inflates the layout, wraps it in a ViewHolder
<code>onBindViewHolder()</code>	Every time a position scrolls into view	Fills the cached views with this item's data
<code>getItemCount()</code>	Whenever the list size is checked	Returns the total number of items

Analogy: `onCreateViewHolder()` builds an empty picture frame (once); `onBindViewHolder()` slides a new photo into that frame (every time it's needed).

Adding Divider Lines

ItemDecoration draws extra visuals around each row

```
DividerItemDecoration divider = new DividerItemDecoration(  
    getApplicationContext(), DividerItemDecoration.VERTICAL);  
recyclerView.addItemDecoration(divider);
```

ItemDecoration is a general hook for drawing on top of or between rows — dividers are the built-in, ready-to-use example.

Just like everything else in RecyclerView, this is opt-in: plain ListView drew a divider by default; RecyclerView leaves that choice to you.

Detecting Item Clicks in RecyclerView

Preferred approach: implement the listener inside the ViewHolder

```
public class MyViewHolder extends RecyclerView.ViewHolder implements View.OnClickListener {

    TextView title;

    public MyViewHolder(View itemView) {
        super(itemView);
        title = itemView.findViewById(R.id.text1);
        itemView.setOnClickListener(this);
    }

    @Override
    public void onClick(View v) {
        int position = getLayoutPosition();
        Toast.makeText(v.getContext(), fruits[position], Toast.LENGTH_SHORT).show();
    }
}
```

ListView vs RecyclerView

A quick side-by-side to help you choose

Feature	ListView	RecyclerView
ViewHolder pattern	Optional (manual)	Required (built-in)
Layout options	Vertical list only	Linear / Grid / Staggered
Item animations	Manual	Built-in support
Dividers	XML attribute	ItemDecoration class
Best for	Simple, short lists	Complex or dynamic lists

Deleting & Moving Items

ItemTouchHelper adds swipe-to-delete and drag-to-reorder

ItemTouchHelper.SimpleCallback listens for two gestures and reports them back to your adapter:

```
new ItemTouchHelper.SimpleCallback(UP | DOWN, LEFT | RIGHT) {  
    public boolean onMove(RecyclerView rv, ViewHolder from, ViewHolder to) {  
  
        adapter.moveItem(from.getAdapterPosition(), to.getAdapterPosition());  
  
        return true;  
    }  
    public void onSwiped(ViewHolder vh, int direction) {  
  
        adapter.deleteItem(vh.getAdapterPosition());  
  
    }  
}
```

```
Attach it with: new ItemTouchHelper(callback).attachToRecyclerView(recyclerView);
```

Adding a New Item to the List

An EditText + Button pair, feeding straight into the adapter

1

MainActivity — button click

```
Button addButton = findViewById(R.id.addButton);
addButton.setOnClickListener(v -> {
    EditText input = findViewById(R.id.fruitName);
    String fruit = input.getText().toString();
    input.setText("");
    adapter.addItem(fruit);
});
```

2

MyAdapter — addItem()

```
void addItem(String fruit) {
    fruits.add(fruit);
    notifyItemInserted(fruits.size() - 1);
}
```

Quick Check

Can You Answer These?

Take 30 seconds — think it through or discuss with a neighbor

Q1 What are the three methods every RecyclerView.Adapter must implement?

Q2 Why does the ViewHolder pattern make scrolling faster?

Q3 What's the difference between onCreateViewHolder() and onBindViewHolder()?

Q4 Name one thing RecyclerView can do that plain ListView can't.

Recap: Session 7

What we learned today

ListView & Adapter

A scrollable row list, driven by
getCount/getItem/getView

RecyclerView

The modern, flexible list widget with
pluggable LayoutManagers

ViewHolder Pattern

Caches findViewById() results once per
row for fast scrolling

onCreate vs onBind

Create builds the row once; Bind fills it
with data every time

ItemTouchHelper

Adds swipe-to-delete and drag-to-reorder
to a RecyclerView

Thank You

Questions?

by Ekramul Haque Amin

Lecturer, DIU